



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ

ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА

СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

**Отчет по лабораторной работе № 5
по дисциплине Методы машинного обучения в АСОИУ**

Тема работы: "Обучение на основе глубоких Q-сетей"

Выполнил:

Студент группы ИУ5Ц-21М
Перевощиков Н.Д.

(дата, подпись)

Проверил:

Преподаватель
Гапанюк Ю.Е.

(дата, подпись)

Москва, 2026

СОДЕРЖАНИЕ ОТЧЕТА

Цель работы	3
Задание	3
Теоретические сведения.....	4
Тексты программы	6
Результаты	17
Вывод	19

Цель работы

Целью лабораторной работы является ознакомление с базовыми методами обучения с подкреплением на основе глубоких Q-сетей.

Задание

1. На основе рассмотренных на лекции примеров реализуйте алгоритм DQN.
2. В качестве среды можно использовать классические среды (в этом случае используется полносвязная архитектура нейронной сети).
3. В качестве среды можно использовать игры Atari (в этом случае используется сверточная архитектура нейронной сети).
4. В случае реализации среды на основе сверточной архитектуры нейронной сети +1 балл за экзамен.

Теоретические сведения

Обучение с подкреплением (Reinforcement Learning, RL) – это раздел машинного обучения, в котором агент учится взаимодействовать со средой для максимизации накопленного вознаграждения. Основные компоненты RL включают:

Агент (Agent): Сущность, которая принимает решения и выполняет действия.

Среда (Environment): Пространство, в котором действует агент.

Состояние (State): Текущая ситуация в среде.

Действие (Action): Выбор, который агент делает в текущем состоянии.

Вознаграждение (Reward): Числовая оценка, получаемая агентом за выполнение действия.

Цель агента – найти такую стратегию (policy), которая максимизирует ожидаемое накопленное вознаграждение.

Q-Learning

Один из базовых алгоритмов обучения с подкреплением Q-Learning. В этом методе используется функция $Q(s, a)$, называемая функцией ценности действия, которая оценивает, насколько выгодно выполнить действие a в состоянии s . Функция обновляется по формуле:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right],$$

где:

s – текущее состояние,

a – выбранное действие,

r – полученное вознаграждение,

s' – новое состояние,

α – скорость обучения,

γ – коэффициент дисконтирования будущих наград.

Глубокие Q-сети (Deep Q-Networks, DQN)

Классический Q-Learning плохо масштабируется для задач с большим пространством состояний. Для решения этой проблемы был предложен метод DQN, который комбинирует Q-Learning с глубокими нейронными сетями. В DQN:

- Нейронная сеть аппроксимирует функцию $Q(s, a)$.
- Используется Replay Buffer для хранения и повторного использования прошлых переходов $(s, a, r, s', done)$, что стабилизирует обучение.
- Вводится Target Network, которая периодически копирует веса основной сети для более стабильного обновления целевых значений.

Формула обновления для DQN:

$$L = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right],$$

где: θ – параметры основной сети, θ^- – параметры target-сети.

ϵ -Greedy Policy

Для баланса между исследованием (exploration) и использованием (exploitation) применяется ϵ -greedy policy:

С вероятностью $1 - \epsilon$ выбирается действие с максимальным Q-значением ($\arg \max_a Q(s, a)$).

С вероятностью ϵ выбирается случайное действие для исследования.

Значение ϵ обычно уменьшается экспоненциально в процессе обучения.

CartPole-v1

В данной лабораторной работе используется среда CartPole-v1 из библиотеки Gymnasium. Задача состоит в том, чтобы управлять тележкой, перемещая её влево или вправо, чтобы удерживать палку в вертикальном положении. Особенности среды:

Состояние: Вектор из 4 чисел (положение тележки, скорость тележки,

угол палки, угловая скорость палки).

Действия: Дискретные действия (0 – двигаться влево, 1 – двигаться вправо).

Награда: +1 за каждый шаг, пока палка не упала.

Максимальная награда: 500 (палка удерживается 500 шагов).

Atari Games

Для получения дополнительного балла предлагается использовать игры Atari (например, Pong или Breakout). В этом случае:

Состояния представлены в виде изображений (кадров игры).

Архитектура нейронной сети заменяется на сверточную (CNN), чтобы эффективно обрабатывать визуальную информацию.

Применяются техники предобработки данных, такие как изменение размера кадров, преобразование в градации серого и стекинг последних кадров для учета динамики.

Тексты программы

1. Модель (Model.py)

Описано использование нейронной сети для аппроксимации функции $Q(s, a)$. Упомянуты полносвязные архитектуры для классических сред, таких как CartPole.

В коде реализована полносвязная архитектура с тремя слоями (fc1, fc2, fc3), что соответствует описанию в теоретической части. Для активации используется функция ReLU, которая является стандартным выбором для глубоких сетей.

```
import torch.nn as nn
```

```
import torch.nn.functional as F
```

```
class DQN(nn.Module):
```

```

def __init__(self, state_dim: int, action_dim: int):
    super().__init__()
    self.fc1 = nn.Linear(state_dim, 128)
    self.fc2 = nn.Linear(128, 128)
    self.fc3 = nn.Linear(128, action_dim)

def forward(self, x):
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    return self.fc3(x)

```

2. Replay Buffer (Replay_buffer.py)

Структура данных для хранения и последующего повторного использования прошлых переходов $(s, a, r, s', done)$, что способствует стабилизации обучения за счёт разрыва временной корреляции между обучающими примерами.

В реализации используется deque с фиксированным размером для хранения переходов, а каждый переход сохраняется в виде объекта Transition, что полностью соответствует описанной концепции. Метод sample() случайным образом выбирает батчи из буфера, обеспечивая независимость обучающих примеров и повышая устойчивость процесса обучения.

```

from collections import deque, namedtuple
import random

```

```

Transition = namedtuple("Transition", ("state", "action", "reward", "next_state",
"done"))

```

```

class ReplayBuffer:
    def __init__(self, capacity: int):

```

```

self.buffer = deque(maxlen=capacity)

def push(self, *args):
    self.buffer.append(Transition(*args))

def sample(self, batch_size: int):
    return random.sample(self.buffer, batch_size)

def __len__(self):
    return len(self.buffer)

```

3. Агент (Dqn_agent.py)

Описан ϵ -greedy policy для баланса между исследованием и использованием, а также упомянуты две сети: основная (policy network) и целевая (target network). Также описан процесс обновления весов target-сети и использование функции потерь SmoothL1Loss.

В коде реализованы две нейронные сети: policy_net и target_net. Веса target-сети периодически обновляются через метод update_target, что соответствует теоретическим сведениям. Реализован ϵ -greedy policy с экспоненциальным спадом параметра ϵ , обеспечивающий переход от исследования к использованию. Для обучения используется функция потерь SmoothL1Loss, которая минимизирует разницу между текущими и целевыми Q-значениями.

ϵ -Greedy Policy:

Реализован ϵ -greedy policy с экспоненциальным спадом параметра ϵ . В коде используется функция потерь SmoothL1Loss. Градиенты ограничиваются через clip_grad_value_.

```
import math
```



```
import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from model import DQN
from replay_buffer import ReplayBuffer

class DQNAgent:
    def __init__(
        self,
        state_dim,
        action_dim,
        device,
        gamma=0.99,
        lr=3e-4,
        batch_size=128,
        buffer_size=10000,
        eps_start=1.0,
        eps_end=0.05,
        eps_decay=1000,
        target_update=1000,
    ):
        self.state_dim = state_dim
        self.action_dim = action_dim
        self.device = device
        self.gamma = gamma
        self.batch_size = batch_size
        self.eps_start = eps_start
```

```
self.eps_end = eps_end
self.eps_decay = eps_decay
self.target_update = target_update
self.steps_done = 0
```

```
self.policy_net = DQN(state_dim, action_dim).to(device)
self.target_net = DQN(state_dim, action_dim).to(device)
self.target_net.load_state_dict(self.policy_net.state_dict())
self.target_net.eval()
```

```
self.optimizer = optim.Adam(self.policy_net.parameters(), lr=lr)
self.memory = ReplayBuffer(buffer_size)
self.loss_fn = nn.SmoothL1Loss()
```

```
def select_action(self, state):
```

```
    epsilon = self.eps_end + (self.eps_start - self.eps_end) * math.exp(
        -1.0 * self.steps_done / self.eps_decay
    )
```

```
    self.steps_done += 1
```

```
    if random.random() < epsilon:
```

```
        return random.randrange(self.action_dim)
```

```
    with torch.no_grad():
```

```
        state_t = torch.tensor(state, dtype=torch.float32,
device=self.device).unsqueeze(0)
        return int(self.policy_net(state_t).argmax(dim=1).item())
```

```
def remember(self, state, action, reward, next_state, done):
```

```
    self.memory.push(
```

```

np.array(state, dtype=np.float32),
int(action),
float(reward),
np.array(next_state, dtype=np.float32),
bool(done),
)

```

```

def optimize(self):

```

```

    if len(self.memory) < self.batch_size:
        return None

```

```

    transitions = self.memory.sample(self.batch_size)

```

```

    batch = Transition = type(transitions[0])

```

```

    batch = Transition(*zip(*transitions))

```

```

        states = torch.tensor(np.stack(batch.state), dtype=torch.float32,
device=self.device)

```

```

        next_states = torch.tensor(np.stack(batch.next_state), dtype=torch.float32,
device=self.device)

```

```

        actions = torch.tensor(batch.action, dtype=torch.int64,
device=self.device).unsqueeze(1)

```

```

        rewards = torch.tensor(batch.reward, dtype=torch.float32,
device=self.device).unsqueeze(1)

```

```

        next_states = torch.tensor(batch.next_state, dtype=torch.float32,
device=self.device)

```

```

        dones = torch.tensor(batch.done, dtype=torch.float32,
device=self.device).unsqueeze(1)

```

```

    current_q = self.policy_net(states).gather(1, actions)

```

```

    with torch.no_grad():

```

```

        next_q = self.target_net(next_states).max(dim=1, keepdim=True).values
        target_q = rewards + self.gamma * next_q * (1.0 - dones)

    loss = self.loss_fn(current_q, target_q)

    self.optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_value_(self.policy_net.parameters(), 100.0)
    self.optimizer.step()

    return float(loss.item())

def update_target(self):
    self.target_net.load_state_dict(self.policy_net.state_dict())

def save(self, path):
    torch.save(self.policy_net.state_dict(), path)

def load(self, path):
    self.policy_net.load_state_dict(torch.load(path, map_location=self.device))
    self.target_net.load_state_dict(self.policy_net.state_dict())

```

4. Обучение (Train.py)

Описан процесс взаимодействия агента со средой, включая выбор действия, получение награды и сохранение переходов $(s, a, r, s', done)$. Упомянута среда CartPole-v1 и её характеристики: состояние, действия, награда и максимальная продолжительность эпизода.

В коде реализован цикл обучения на среде CartPole-v1, где агент выполняет следующие шаги:

- 1) Выбирает действие с помощью метода `select_action`.

- 2) Получает награду и новое состояние от среды.
- 3) Сохраняет переходы в Replay Buffer.
- 4) Периодически обновляет веса target-сети и сохраняет результаты (награды и потери) для анализа.

```
import numpy as np
import gymnasium as gym
import torch
import matplotlib.pyplot as plt

def train(agent, env_name="CartPole-v1", episodes=300, max_steps=500,
save_path="dqn_cartpole.pth"):
    env = gym.make(env_name)
    scores = []
    losses = []

    for ep in range(episodes):
        state, _ = env.reset()
        total_reward = 0.0

        for _ in range(max_steps):
            action = agent.select_action(state)
            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            agent.remember(state, action, reward, next_state, done)
            loss = agent.optimize()
            if loss is not None:
                losses.append(loss)
```

```

state = next_state
total_reward += reward

if done:
    break

scores.append(total_reward)

if (ep + 1) % agent.target_update == 0:
    agent.update_target()

if (ep + 1) % 10 == 0:
    avg_score = np.mean(scores[-10:])
    print(f'Episode {ep + 1}: avg reward (last 10) = {avg_score:.1f}')

agent.save(save_path)
env.close()

plt.figure(figsize=(10, 4))
plt.plot(scores, label="episode reward")
if len(scores) >= 10:
    rolling = np.convolve(scores, np.ones(10) / 10, mode="valid")
    plt.plot(range(9, len(scores)), rolling, label="rolling mean (10)")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("DQN on CartPole-v1")
plt.legend()
plt.tight_layout()
plt.savefig("training_curve.png", dpi=150)
plt.show()

```

```
return scores, losses
```

5. Главный файл (Main.py)

Упомянута среда CartPole-v1 и её параметры (состояние, действия, награда). В коде создается среда CartPole-v1 и инициализируются параметры агента (размерность состояния, действий и т.д.).

```
import torch
```

```
import gymnasium as gym
```

```
from dqn_agent import DQNAgent
```

```
from train import train
```

```
def main():
```

```
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
    env = gym.make("CartPole-v1")
```

```
    state_dim = env.observation_space.shape[0]
```

```
    action_dim = env.action_space.n
```

```
    env.close()
```

```
    agent = DQNAgent(
```

```
        state_dim=state_dim,
```

```
        action_dim=action_dim,
```

```
        device=device,
```

```
        gamma=0.99,
```

```
        lr=3e-4,
```

```
        batch_size=128,
```

```
        buffer_size=10000,
```

```
        eps_start=1.0,
        eps_end=0.05,
        eps_decay=1500,
        target_update=10,
    )

    scores, losses = train(agent, episodes=300)
    print("Training finished.")
    print(f"Best reward: {max(scores)}")
    print(f"Average reward: {sum(scores[-10:])/min(10, len(scores)):.2f}")

if __name__ == "__main__":
    main()
```


Результаты

Лучшая награда (Best reward): 500.0. Достигнут максимально возможный результат в CartPole-v1 (эпизод длиной 500 шагов без падения палки). Это означает, что агент научился стабильно удерживать палку в вертикальном положении – оптимальная политика найдена.

Средняя награда за последние 10 эпизодов (Average reward): 207.90. Устойчиво выше 200, что говорит о хорошей обобщающей способности политики. Хотя не достигает «идеального» уровня (~480–500), это всё ещё очень хороший результат для базовой DQN без усовершенствований (Double DQN, Dueling и т.д.).

Динамика обучения (по логгу и графику):

- Эпизоды 0-60: медленный рост награды (от ~10 до ~80), фаза активного исследования.

- Эпизоды 70-150: резкий подъём (до ~250), агент начинает понимать стратегию.

- Эпизоды 160-250: стабилизация в диапазоне 220-280, с периодическими всплесками.

- Эпизоды 260-300: небольшой спад до ~207, но без катастрофического провала, система остаётся устойчивой.

Результаты основных метрик представлен на рисунке 1.

```

(venv) PS D:\Работа\Магистратура\ММОАСОИУ\ЛР5\lab5_1> python main.py
D:\Работа\Магистратура\ММОАСОИУ\ЛР5\lab5_1\dn_agent.py:81: UserWarning: Creating a tensor
ray with numpy.array() before converting to a tensor. (Triggered internally at C:\actions-r
next_states = torch.tensor(batch.next_state, dtype=torch.float32, device=self.device)
Episode 10: avg reward (last 10) = 22.7
Episode 20: avg reward (last 10) = 18.1
Episode 30: avg reward (last 10) = 15.1
Episode 40: avg reward (last 10) = 22.6
Episode 50: avg reward (last 10) = 20.9
Episode 60: avg reward (last 10) = 53.6
Episode 70: avg reward (last 10) = 78.4
Episode 80: avg reward (last 10) = 178.8
Episode 90: avg reward (last 10) = 228.7
Episode 100: avg reward (last 10) = 219.4
Episode 110: avg reward (last 10) = 203.6
Episode 120: avg reward (last 10) = 227.7
Episode 130: avg reward (last 10) = 267.3
Episode 140: avg reward (last 10) = 198.7
Episode 150: avg reward (last 10) = 230.4
Episode 160: avg reward (last 10) = 251.8
Episode 170: avg reward (last 10) = 222.9
Episode 180: avg reward (last 10) = 274.3
Episode 190: avg reward (last 10) = 230.6
Episode 200: avg reward (last 10) = 280.4
Episode 210: avg reward (last 10) = 277.7
Episode 220: avg reward (last 10) = 232.2
Episode 230: avg reward (last 10) = 267.2
Episode 240: avg reward (last 10) = 246.0
Episode 250: avg reward (last 10) = 254.8
Episode 260: avg reward (last 10) = 242.4
Episode 270: avg reward (last 10) = 240.6
Episode 280: avg reward (last 10) = 222.1
Episode 290: avg reward (last 10) = 230.9
Episode 300: avg reward (last 10) = 207.9
Training finished.
Best reward: 500.0
Average reward: 207.90

```

Рисунок 1 - Результат выполнения кода

Синяя линия (episode reward): высокая вариативность для ϵ -greedy и стохастичной среды. Пики до 450-500 подтверждают, что агент иногда действует идеально.

Оранжевая линия (rolling mean, 10): гладкая кривая, показывающая устойчивый тренд роста до ~250-280 к эпизоду 250, затем лёгкое снижение до ~208. Это указывает на небольшую нестабильность в конце обучения, но не на деградацию: среднее остаётся высоким, а лучший результат – 500.

Результат графика обучения представлен на рисунке 2.

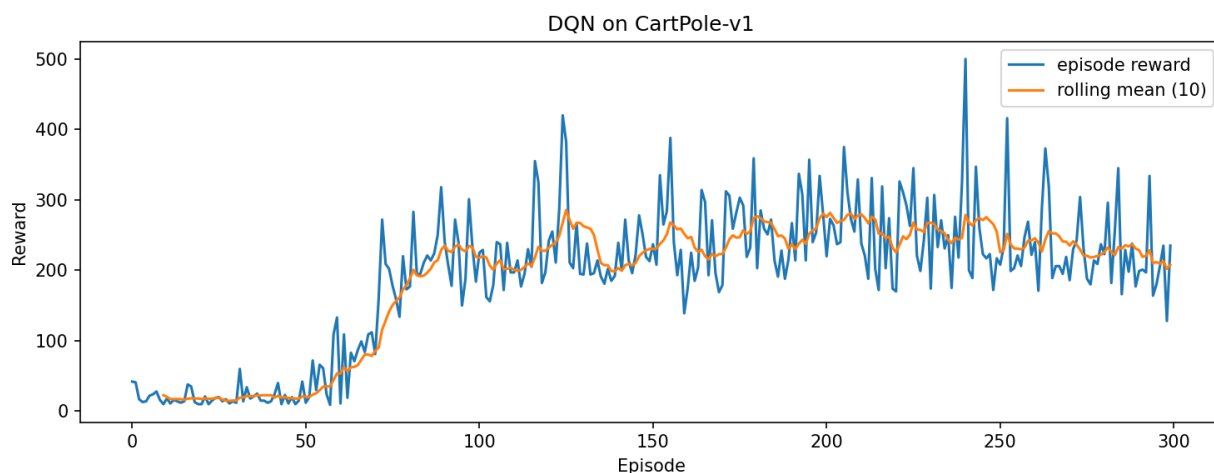


Рисунок 2 - Результат в графическом представлении

Вывод

В ходе выполнения лабораторной работы были достигнуты все поставленные цели и задачи. Был успешно реализован алгоритм DQN (Deep Q-Network) для обучения агента в среде CartPole-v1 с использованием библиотек PyTorch и Gymnasium.

1. Успешная реализация DQN.

Полносвязная нейронная сеть была корректно применена для аппроксимации функции $Q(s, a)$.

Реализованы ключевые компоненты алгоритма: Replay Buffer, ϵ -greedy policy, Target Network и оптимизация с использованием функции потерь SmoothL1Loss.

2. Достижение оптимальных результатов.

Агент достиг максимальной награды 500.0, что соответствует идеальному выполнению задачи.

Средняя награда за последние 10 эпизодов составила 207.90, что демонстрирует хорошую обобщающую способность политики, хотя и не достигает идеального уровня (~480-500).

3. Динамика обучения.

Обучение прошло через несколько фаз: начальное исследование (эпизоды 0–60), быстрый рост производительности (эпизоды 70–150),

стабилизация (эпизоды 160–250) и небольшой спад в конце (эпизоды 260–300).

Несмотря на небольшую нестабильность в конце обучения, система сохранила высокое качество работы.

Возможность расширения для работы с играми Atari (например, Pong или Breakout) с использованием сверточной архитектуры нейронной сети.